

Capítulo 14

Sft

Construindo um Modelo de Linguagem do Zero

Curso bilingue inspirado no LLM101n

Gerado em 10/08/2026

Capítulo 14 — Finetuning I: SFT

Objetivo de aprendizagem: transformar um modelo que **continua texto** num modelo que **responde e para**. Implementar o *supervised finetuning* do zero: tokens especiais, expansão de vocabulário, máscara de loss — e medir a mudança com um número binário, não com uma impressão de leitura.

Pré-requisitos: Capítulos 11 (o modelo treinado) e 12 (geração).

Arquivos:

- `preparar_sft.py` — monta o dataset de instrução a partir do corpus
- `sft.py` — o finetuning: expande o vocabulário e treina com máscara
- `avaliar.py` — a medição que define se funcionou
- `exercicios.md` — exercícios

Uma promessa que este capítulo não faz. Um modelo de 2,2 M parâmetros treinado em 1,6 MB de Machado **não vira um assistente**. Ele não vai responder perguntas, não vai seguir instruções complexas, não vai ficar inteligente. Fingir o contrário estragaria o capítulo.

O que ele **consegue** aprender é exatamente o mecanismo que separa um modelo-base de um modelo de chat — e que dá para medir com honestidade nesta escala.

1. O que falta a um modelo-base

O modelo do Capítulo 11 sabe uma coisa: dado um texto, prever o próximo token. Peça a ele que continue "Havia em mim" e ele continua. Continua, e continua, e continua — até você mandar parar.

Ele não sabe terminar. Não é uma limitação de qualidade; é que a tarefa de pré-treino nunca tem um fim. O corpus é um fluxo contínuo de texto, e o modelo aprendeu a modelar esse fluxo.

Um modelo de chat faz três coisas que este não faz:

Capacidade	Modelo-base	Modelo de chat
Distingue quem falou	não	sim (papéis)
Sabe onde a resposta começa	não	sim
Sabe onde a resposta termina	não	sim

A terceira é a mais fácil de medir e a mais fundamental. É a que este capítulo persegue.

2. Tokens que não são texto

Para marcar as fronteiras, precisamos de símbolos que o texto **não possa produzir**:

```
PEDIDO, RESPOSTA, FIM = 1024, 1025, 1026
```

O vocabulário do Capítulo 11 vai de 0 a 1023. Esses três ids são novos, e nenhuma sequência de caracteres do corpus os gera.

Por que não usar uma string, tipo `###RESPOSTA###`? Porque o próprio texto poderia contê-la. Um token especial é uma garantia estrutural: não existe entrada do usuário que produza o id 1025. Quando você ouvir falar de *prompt injection* via delimitadores, é exatamente essa a diferença.

O formato de cada exemplo:

```
<|pedido|> {contexto} <|resposta|> {continuação} <|fim|>
```

3. Aumentar o vocabulário de um modelo já treinado

O modelo tem 1.024 saídas e precisa de 1.027. Isso significa esticar **duas** matrizes: a tabela de embeddings (entrada) e a camada final (saída).

A parte mecânica é trivial. A decisão que importa é: **com o que preencher as linhas novas?**

```
te.weight.data[:velho] = m.te.weight.data # o que já existia
te.weight.data[velho:] = m.te.weight.data.mean(0) # os tokens novos
```

Zeros parecem naturais e são ruins na camada de saída. Um logit zero para o token novo não significa "neutro" — significa um valor específico numa distribuição já treinada, e ele compete de igual para igual com os tokens mais improváveis. A rede leva tempo para separá-lo.

Inicializar com a **média das linhas existentes** faz o token novo nascer como "um token médio": ele herda a escala e a estatística do que já está lá, e se afasta dali conforme aprende. É o que se faz na prática, e é mais estável.

Custo total: **1.155 parâmetros a mais** — 2.196.352 -> 2.197.507. Um aumento de 0,05% que muda o que o modelo é capaz de expressar.

4. A máscara de loss

Cada exemplo tem duas partes com papéis diferentes:

```
<|pedido|> {contexto} <|resposta|> {continuação} <|fim|>
\_____ vem do usuário _____/ \___ o modelo produz ___/
```

O modelo não precisa aprender a escrever o pedido — o pedido chega pronto. Então a loss é calculada **só na resposta**:

```
Y[i, :] = IGNORAR # -100 em tudo
Y[i, ini_resposta:fim] = seq[ini_resposta+1:] # exceto na resposta
...
F.cross_entropy(logits, alvos, ignore_index=-100)
```

O `ignore_index=-100` é do PyTorch e sai de graça: aquelas posições simplesmente não entram na média.

Quanto isso muda o resultado é a Seção 7 — e a resposta não é a que eu esperava.

5. O resultado: o modelo aprende a parar

600 passos, `lr = 3e-4` (três vezes menor que a do pré-treino: não queremos destruir o que já existe). Menos de cinco minutos.

Avaliação em 40 pedidos da validação, com orçamento de 120 tokens:

Modelo	Taxa de parada	Comprimento mediano
modelo-base	0%	nunca para
SFT (com máscara)	100%	34 tokens

0% para 100%. E o zero não é "o modelo erra sempre" — é que o token `<|fim|>` **não existe** no vocabulário do modelo-base. Ele não tem como acertar.

Essa é a distância que o SFT atravessa: não uma melhora gradual, mas uma capacidade que antes não estava lá.

Por que medir isto, e não "a qualidade das respostas"

A tentação é gerar duas amostras, ler, e declarar que melhorou. Numa escala destas isso não significa nada: o modelo escreve mal antes e depois.

E o [Capítulo 12](#) já provou o argumento da forma mais dura possível — lá, um bug rebaixou o Transformer ao nível de um bigrama, e **não dava para perceber lendo o texto**.

Uma taxa de parada é binária, objetiva e não depende de julgamento. Prefira sempre a métrica que não precisa da sua opinião.

6. O erro que eu cometi montando os dados

A primeira versão deste capítulo usava resposta de **tamanho fixo**: 40 tokens, sempre. O resultado:

Modelo	Taxa de parada	Comprimento mediano
SFT (com máscara)	100%	40

SFT (sem máscara)	100%	40
-------------------	------	----

Cem por cento nos dois, mediana cravada em 40. Parecia um sucesso.

Era um temporizador. Todos os 8.000 exemplos tinham exatamente 66 tokens, então a única regra consistente nos dados era *posicional*: "pare na posição 66". O modelo aprendeu a contar, não a concluir — e aprendeu perfeitamente, porque era a regra que estava lá.

É o modo de falha mais característico do SFT. O modelo aprende a regularidade que os seus dados de fato contêm, não a que você pretendia ensinar. Se todas as suas respostas começam com "Claro!", ele aprende a dizer "Claro!". Se todas têm o mesmo tamanho, ele aprende o tamanho.

A correção: a resposta agora vai até o primeiro **fim de frase** depois de um mínimo. Os comprimentos passam a variar de 39 a 82 tokens, com 44 valores distintos — e parar exige olhar o texto.

7. A máscara importa? Menos do que dizem — e dá para saber quando

Todo tutorial de SFT recomenda mascarar o prompt. Medindo, com pedido de 24 tokens e resposta de ~34:

Variante	Loss na resposta	Taxa de parada
com máscara	4,0030	100%
sem máscara	3,9995	100%

Nenhuma diferença. A versão sem máscara é até marginalmente melhor, dentro do ruído.

Eu tinha escrito, antes de medir, que treinar no pedido "gasta capacidade com uma tarefa que ninguém pediu". A medição não sustenta isso — nesta configuração.

Mas um resultado nulo não encerra a questão; ele a reformula. **Sob que condição a máscara passaria a importar?**

A hipótese vem da própria definição: a loss sem máscara é a **média sobre todas as posições**. Com o pedido curto, ele é uma fração pequena e quase não dilui o sinal da resposta. Com o pedido **longo**, ele domina a média.

Medindo, com `e2_quando_a_mascara_importa.py`:

Pedido/resposta	% de pedido	Penalidade de não mascarar
24 / ~34	42%	-0,0066 (mascarar é pior)
64 / ~20	72%	+0,0136

O sinal **inverte** conforme o pedido cresce, como a hipótese previa. Repetindo a configuração de pedido longo com **seis sementes**: 6 de 6 na mesma direção, média **+0,0199 ± 0,0112**.

(!) **Mas olhe a magnitude**: 0,0199 sobre uma loss de 3,95 é **0,5%**. O efeito é real e pequeno. "Estatisticamente detectável" e "importante na prática" são coisas diferentes, e este número é o primeiro sem ser o segundo — nesta escala.

Por que isso importa na prática: em SFT de verdade a proporção é justamente a desfavorável. Um prompt de sistema com instruções longas, mais o histórico da conversa, mais a pergunta — tudo isso costuma ser muito maior que a resposta. A recomendação padrão está certa **no regime em que foi formulada**, e este capítulo não reproduz esse regime por acidente de escala.

E um erro meu que virou a parte mais útil do exercício

Ao analisar as primeiras três sementes, eu comparei a diferença entre variantes (0,0175) com a variação da **mesma** variante entre sementes (0,0348) e concluí: *"o ruído supera o efeito, a tendência não está estabelecida"*.

O critério estava errado para o desenho do experimento. As duas variantes rodam com a **mesma semente** — é um experimento **pareado**. Nele, a variação que a semente causa afeta os dois lados e **se cancela na diferença**. Comparar o efeito com a variação absoluta joga fora exatamente a vantagem de ter pareado.

O mesmo dado, os dois critérios:

Critério	Números	Veredito
não pareado	ruído 0,0348 > efeito 0,0199	"não estabelecido"
pareado	6/6 na mesma direção, média > desvio	estabelecido

Conclusões opostas, decididas só pela estatística escolhida.

É o segundo caso deste curso em que a **medição estava certa e a leitura não**. O outro foi o [Capítulo 13](#), onde uma métrica global marcava 1,3% de erro enquanto 48% das linhas estavam destruídas.

Escolher a métrica, e escolher o teste, são decisões tão determinantes quanto rodar o experimento — e passam despercebidas porque parecem detalhes técnicos depois do trabalho "de verdade".

8. O que o SFT é, e o que não é

Vale terminar com a distinção que o capítulo inteiro sustenta:

O SFT não acrescenta conhecimento. Ele muda o que o modelo faz com o conhecimento que já tem.

O modelo depois do finetuning não sabe nada de Machado que não soubesse antes — são os mesmos pesos, ajustados por 600 passos com learning rate pequena. O que mudou foi o **comportamento**: ele agora reconhece um formato, sabe onde a sua vez começa e sabe onde ela termina.

É por isso que finetuning não conserta um modelo-base ruim, e é por isso que "vou fazer finetuning para ensinar os meus dados ao modelo" quase sempre decepciona. Conhecimento entra no pré-treino, ou entra pelo contexto. O SFT molda a **forma**.

9. Resumo do capítulo

- Um modelo-base **não sabe terminar** — a tarefa de pré-treino nunca tem fim.
- **Tokens especiais** funcionam como delimitadores porque o texto não pode produzi-los.
- Aumentar o vocabulário exige esticar embeddings e camada de saída; inicialize as linhas novas com a **média**, não com zeros.
- A **máscara de loss** sai de graça com `ignore_index=-100` — e importa quando o pedido domina as posições, não sempre.
- Meça com um número **binário** (taxa de parada), não lendo amostras.
- **Cuidado com regularidades acidentais nos dados**: se todas as respostas têm o mesmo tamanho, o modelo aprende o tamanho.
- SFT muda **comportamento**, não conhecimento.

Próximo capítulo

Capítulo 15 — Finetuning II: RL. O SFT ensina o modelo a imitar respostas que alguém escreveu. E quando não há resposta certa para imitar — só respostas melhores e piores? Aí é preciso otimizar contra uma **preferência**, e não contra um alvo.

Exercícios — Capítulo 14 (SFT)

Faça na ordem. Soluções comentadas em [solucoes/](#) — tente antes de olhar.

Rode `python preparar_sft.py` e `python sft.py` antes dos exercícios. Juntos levam menos de seis minutos.

E1 — Tokens especiais e vocabulário (aquecimento)

Sem rodar:

1. Por que `<|fim|>` precisa ser um **id novo** em vez de uma string como `###FIM###`? Descreva um caso concreto em que a string falharia.
 2. Ao aumentar o vocabulário de 1.024 para 1.027, quantos parâmetros novos aparecem? Faça a conta com `n_embd = 192`, lembrando das **duas** matrizes e do bias.
 3. As linhas novas são inicializadas com a **média** das existentes. O que aconteceria com zeros na camada de saída? E com valores aleatórios grandes?
-

E2 — Quando a máscara importa (importante)

O capítulo mediu que mascarar o pedido **não faz diferença** com pedido de 24 tokens e resposta de ~34.

1. Antes de rodar: escreva por que você espera que a máscara importe, e o que na sua explicação depende do **tamanho relativo** do pedido.
 2. Rode `solucoes/e2_quando_a_mascara_importa.py`, que varia a proporção. A partir de qual fração de pedido a penalidade aparece?
 3. Um resultado nulo ("não faz diferença") e um resultado negativo ("faz diferença ao contrário") pedem reações diferentes. Qual dos dois você obteve no caso de pedido curto, e o que cada um autorizaria você a concluir?
-

E3 — Quebre os dados de propósito (importante)

Volte o `preparar_sft.py` para resposta de **tamanho fixo** (era `TAM_RESPOSTA = 40`).

1. Treine e meça a taxa de parada e o comprimento **mediano**. O que você vê?
 2. A taxa de parada é 100% nos dois casos. Isso significa que a versão de tamanho fixo é igualmente boa? Que métrica separa as duas?
 3. Generalize: se todas as respostas do seu conjunto de SFT comessem com "Claro!", o que o modelo aprenderia? Dê mais dois exemplos de regularidade acidental que apareceriam num conjunto real.
-

E4 — A learning rate do finetuning

O `sft.py` usa `lr = 3e-4`, três vezes menor que a do pré-treino.

1. Rode com `1e-3` (a do pré-treino) e com `3e-5`. Meça a loss na resposta e a taxa de parada nas três.
 2. Com learning rate alta, o que acontece com a capacidade do modelo de escrever português comum? (Sugestão: gere texto **sem** o formato de instrução e compare com o modelo-base.)
 3. O fenômeno tem nome — *catastrophic forgetting*. Explique por que finetuning com poucos dados e learning rate alta o provoca.
-

E5 — Quantos exemplos bastam?

1. Treine com 500, 2.000 e 8.000 exemplos, mantendo os passos fixos. Como muda a taxa de parada? E a loss?
 2. Compare com o Capítulo 11, onde mais dados sempre ajudaram. Por que a curva aqui é diferente?
 3. Na literatura de SFT há resultados mostrando que **mil exemplos bem escolhidos** batem dezenas de milhares aleatórios. O que a sua medição sugere sobre o porquê?
-

E6 — O modelo esqueceu de escrever?

1. Meça a loss do modelo **depois do SFT** no conjunto de validação **original** do Capítulo 11 (texto corrido, sem formato de instrução). Compare com o modelo-base.
 2. O SFT melhorou, piorou ou não mexeu na capacidade de modelar português comum?
 3. Este é o *alignment tax*: o custo, na capacidade original, de ensinar um comportamento novo. Ele é inevitável? O que reduziria o custo?
-

E7 — Um formato de conversa de verdade (desafio)

O formato deste capítulo tem um turno só. Modelos de chat têm vários.

1. Estenda o formato para `<|pedido|> A <|resposta|> B <|pedido|> C <|resposta|> D <|fim|>`. O que muda na máscara?
2. Onde entra o `<|fim|>` — no fim de cada resposta ou só no fim da conversa? Qual das duas escolhas permite a geração parar no turno certo?
3. Modelos reais usam um token de fim **por turno** (`<|im_end|>` e afins) e param nele. Implemente assim e verifique que a geração para no fim do primeiro turno, não da conversa inteira.